



DOCTORADO EN
**INTELIGENCIA
ARTIFICIAL**



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA



UCSC



UNIVERSIDAD DEL BÍO-BÍO

Projective Geometry

Doctorado en Inteligencia Artificial

Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

1 Camera modeling & Calibration

- Perspective projection
- Focal Length
- Camera-Intrinsic Parameters
- Calibration
- Camera-Extrinsic Parameters
- Camera Rotation

2 Direct Linear Transform (DLR) Algorithm

3 Random Sampling with Consensus (RANSAC)

- RANSAC para homografías
- Image Stitching

Camera modeling & Calibration

Una cámara está calibrada si conocemos las transformaciones que relacionan las coordenadas del mundo en 3D con las coordenadas de píxeles dentro de la cámara.

Esas transformaciones implican dos tipos de parámetros:

Una cámara está calibrada si conocemos las transformaciones que relacionan las coordenadas del mundo en 3D con las coordenadas de píxeles dentro de la cámara.

Esas transformaciones implican dos tipos de parámetros:

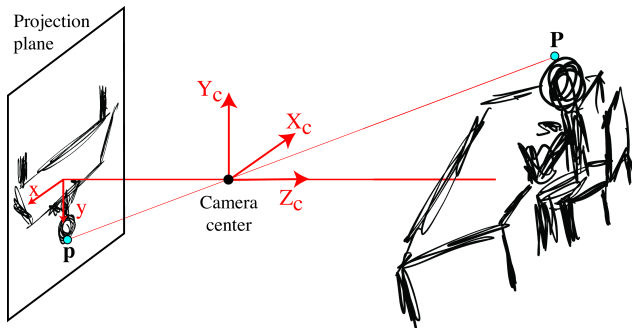
1. **Parámetros intrínsecos:** que dependen de la propia cámara.
2. **Parámetros extrínsecos:** Parámetros que dependen de la ubicación física de la cámara en el mundo.

Proyección perspectiva y coordenadas homogéneas

La proyección perspectiva es no lineal y relaciona la división por Z-location.

Un punto 3D, $\mathbf{P} = [X, Y, Z]^T$, se proyecta en la imagen con coordenadas $[x, y]^T$, donde $x = \frac{fX}{Z}$, y $y = \frac{fY}{Z}$, y f es el focal length.

- No es una transformación lineal... Geometría proyectiva y coordenadas homogéneas simplifican las relaciones algebraicas.



Coordenadas homogéneas $\mathbf{P} = (X, Y, Z, 1)^T$

- Permiten expresar la proyección como multiplicación matricial

K is the perspective projection transformation matrix.

$$\mathbf{KP} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \begin{bmatrix} fX \\ fY \\ Z \end{bmatrix} = \lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\mathbf{KP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1/f & 0 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \begin{bmatrix} fX \\ fY \\ Z \end{bmatrix}$$

- Podemos comprobar que la matriz **K**, multiplicada por el vector que describe el punto 3D **P**, devuelve la posición de su proyección en perspectiva

¹**K** permite mapear coordenadas 3D de cámara a píxeles, incorporando los parámetros intrínsecos.

K is the perspective projection transformation matrix.

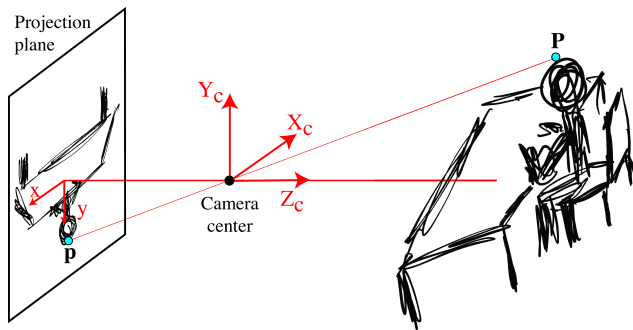
$$\mathbf{KP} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \begin{bmatrix} fX \\ fY \\ Z \end{bmatrix} = \lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\mathbf{KP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1/f & 0 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \begin{bmatrix} fX \\ fY \\ Z \end{bmatrix}$$

- Podemos comprobar que la matriz **K**, multiplicada por el vector que describe el punto 3D **P**, devuelve la posición de su proyección en perspectiva
- ¿Por qué es importante comprobar que la matriz intrínseca de la cámara **K**, al multiplicarse por un punto 3D **P**, permite obtener su proyección en la imagen?

¹**K** permite mapear coordenadas 3D de cámara a píxeles, incorporando los parámetros intrínsecos.

Proyección en coordenadas homogéneas



Al transformar el vector del producto de **coordenadas homogéneas** a **coordenadas heterogéneas**, se obtiene:

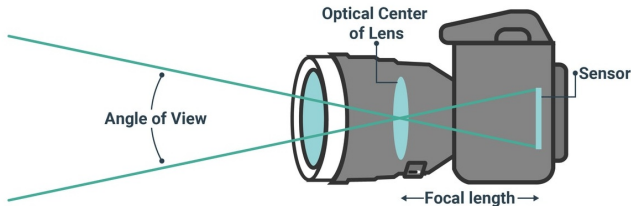
$$\begin{bmatrix} fX \\ fY \\ Z \end{bmatrix} \rightarrow \left(\frac{fX}{Z}, \frac{fY}{Z} \right) = (cx, y) = \mathbf{p},$$

lo que demuestra que la matriz **K**, al multiplicarse por las coordenadas homogéneas de un punto 3D, **P**, **da como resultado las coordenadas homogéneas de la proyección en perspectiva de ese punto 3D, p.**

Distancia focal

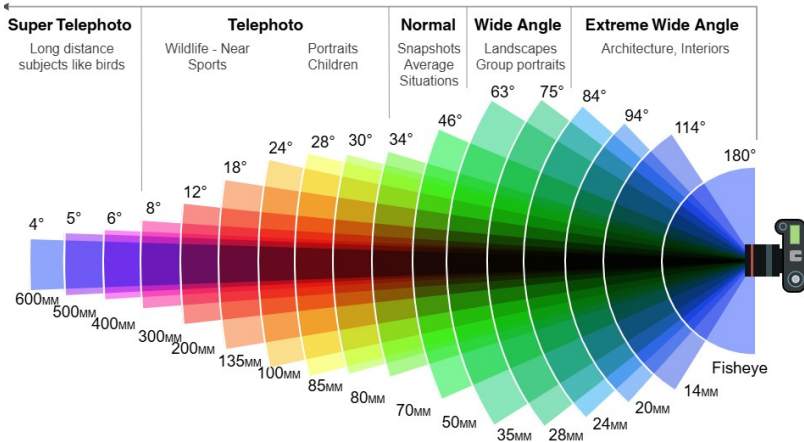
Es la distancia (en mm) entre el centro óptico de la lente y el sensor cuando el sujeto está enfocado al infinito. Determina el campo de visión y el zoom de la imagen:

- ▶ Distancia focal corta (ej. 18mm) → Ángulo amplio (gran angular).
- ▶ Distancia focal larga (ej. 200mm) → Ángulo estrecho (teleobjetivo)

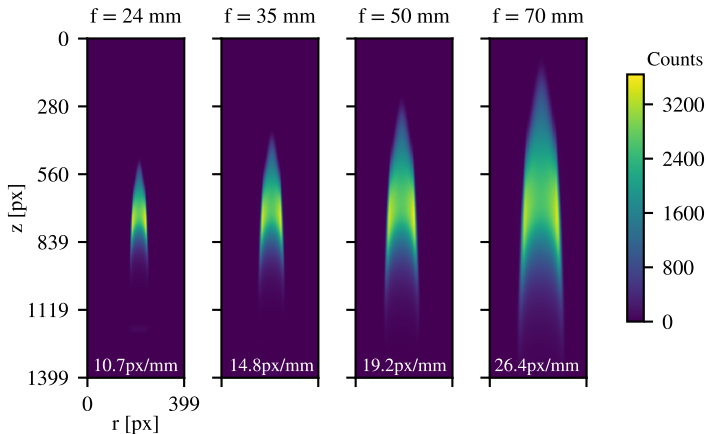


FOCAL LENGTH AND ANGLE OF VIEW

THE DARKROOM.com

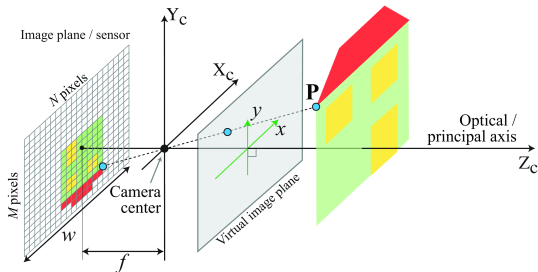


Resolución espacial [px/m]...



From meters to pixels

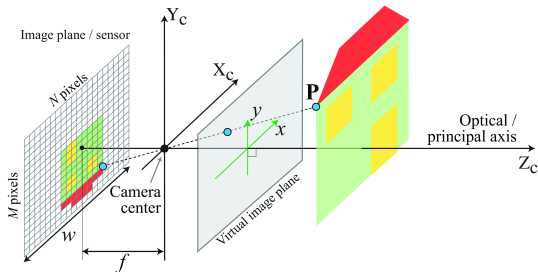
La figura ilustra el proceso de formación de la imagen y su muestreo por parte del sensor.



Se debe tener en cuenta la proyección en perspectiva del punto, P , descrito en coordenadas de la cámara, a coordenadas de píxeles en el plano del sensor de la cámara.

From meters to pixels

La figura ilustra el proceso de formación de la imagen y su muestreo por parte del sensor.



Se debe tener en cuenta la proyección en perspectiva del punto, P , descrito en coordenadas de la cámara, a coordenadas de píxeles en el plano del sensor de la cámara.

$$\mathbf{K} = \begin{bmatrix} \alpha & 0 & 0 & 0 \\ 0 & \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad \alpha = fN/w,$$

donde f es la distancia focal, N es el width de la imagen en píxeles, N/w es la resolución, donde w es el width del sensor en metros.

From meters to pixels

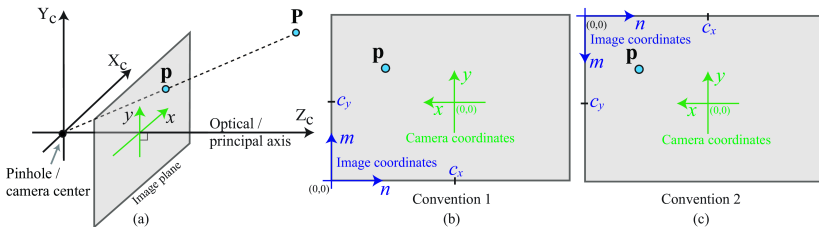
- El centro de la imagen no necesariamente está en $[0, 0]^T$
- Aplicar una traslación para que el eje óptico proyecte en (c_x, c_y)

Para una imagen de tamaño $N \times M$:

$$c_x = \frac{N}{2}, \quad c_y = \frac{M}{2}$$

Matriz intrínseca:

$$\mathbf{K} = \begin{bmatrix} \alpha & 0 & c_x & 0 \\ 0 & \alpha & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$



- El centro de la imagen no necesariamente está en $[0, 0]^T$

² α_1 y α_2 son diferentes si el pixel no es cuadrado

From meters to pixels

- El centro de la imagen no necesariamente está en $[0, 0]^T$
- Aplicar una traslación para que el eje óptico proyecte en (c_x, c_y)

Para una imagen de tamaño $N \times M$:

$$c_x = \frac{N}{2}, \quad c_y = \frac{M}{2}$$

Matriz intrínseca:

$$\mathbf{K} = \begin{bmatrix} \alpha & 0 & c_x & 0 \\ 0 & \alpha & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

La transformación final de las coordenadas 3D, $[X, Y, Z]^T$, a los píxeles de la imagen, $[n, m]^T$, tiene la forma²:

$$\begin{bmatrix} \alpha_1 & 0 & c_x & 0 \\ 0 & \alpha_2 & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} \alpha_1 X/Z + c_x \\ \alpha_2 Y/Z + c_y \end{bmatrix} = \begin{bmatrix} n \\ m \end{bmatrix}$$

² α_1 y α_2 son diferentes si el pixel no es cuadrado

- Estimar parámetros intrínsecos de la cámara

Parámetros desconocidos: **distancia focal y tamaño del sensor.**



1. Tablero de ajedrez 8×10
2. Cada celda: 2×2 cm
3. Ancho total: $W = 20$ cm
4. Distancia cámara-objeto: $Z \approx 31$ cm
5. Imagen capturada, ancho en píxeles: $L = 2002$

Relación geométrica:

$$\alpha = \frac{ZL}{W} = \frac{31 \times 2002}{20} = 3103,1$$

- α corresponde a la escala (relacionada con la distancia focal)

Estimación de parámetros intrínsecos

$$\alpha = \frac{ZL}{W} = \frac{31 \times 2002}{20} = 3103,1$$

Centro de imagen:

$$c_x = \frac{3024}{2} = 1512, \quad c_y = \frac{4032}{2} = 2016$$

Matriz intrínseca:

$$\mathbf{K} = \begin{bmatrix} 3103,1 & 0 & 1512 & 0 \\ 0 & 3103,1 & 2016 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

³Los parámetros intrínsecos obtenidos a partir de las especificaciones del fabricante (e.g., distancia focal y tamaño de sensor) corresponden a un **modelo ideal de cámara tipo pinhole**. En contraste, la calibración geométrica estima estos parámetros a partir de datos observados, capturando efectos reales como distorsión de lente y desviaciones del centro óptico.

Estimación de parámetros intrínsecos

$$\alpha = \frac{ZL}{W} = \frac{31 \times 2002}{20} = 3103,1$$

Centro de imagen:

$$c_x = \frac{3024}{2} = 1512, \quad c_y = \frac{4032}{2} = 2016$$

Matriz intrínseca:

$$\mathbf{K} = \begin{bmatrix} 3103,1 & 0 & 1512 & 0 \\ 0 & 3103,1 & 2016 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Ejemplo: Estimación desde especificaciones de un iPhone 13 pro (usando el wide-angle len $f = 5,7mm$, sensor size $7,6 \times 5,7mm$, y image size 4032×3024)

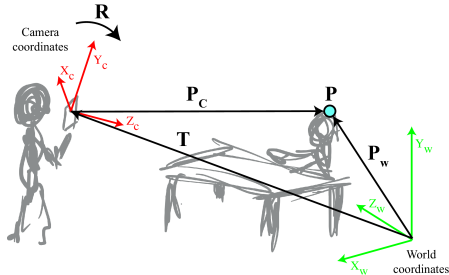
$$\alpha = \frac{f \cdot resolution}{sensor_size} = \frac{4032mm \times 5,7mm}{7,6mm} = 3024$$

Estimación cercana, pero limitada por simplificaciones del modelo.

³Los parámetros intrínsecos obtenidos a partir de las especificaciones del fabricante (e.g., distancia focal y tamaño de sensor) corresponden a un **modelo ideal de cámara tipo pinhole**. En contraste, la calibración geométrica estima estos parámetros a partir de datos observados, capturando efectos reales como distorsión de lente y desviaciones del centro óptico.

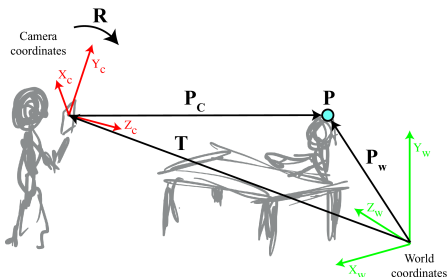
Camera-Extrinsic Parameters

- Se supone que el origen de la cámara se encuentra en el origen del sistema de coordenadas global, con el eje óptico alineado con el eje Z del sistema global.
- En configuraciones donde la cámara se coloca en una ubicación arbitraria alejada del sistema de coordenadas global.



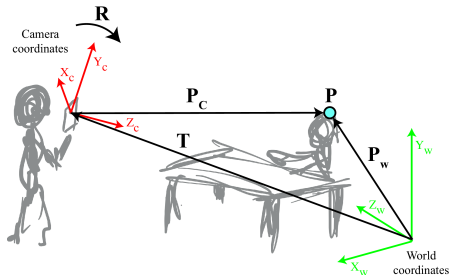
Camera-Extrinsic Parameters

- ▶ Se supone que el origen de la cámara se encuentra en el origen del sistema de coordenadas global, con el eje óptico alineado con el eje Z del sistema global.
- ▶ En configuraciones donde la cámara se coloca en una ubicación arbitraria alejada del sistema de coordenadas global.



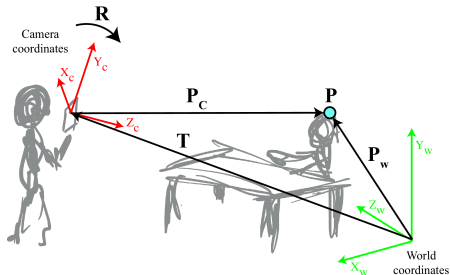
- ▶ El sistema de coordenadas del mundo se ubica de tal forma que el origen se encuentre en el suelo y los ejes Z_w y X_w estén contenidos en el plano del suelo y sean paralelos a los dos ejes definidos por la mesa.
- ▶ El eje Y_w es perpendicular al suelo.
- ▶ Supongamos que el centro de nuestra cámara está desplazado por el vector T y que los ejes R^T están girados un ángulo con respecto al sistema de coordenadas del mundo.

De coordenadas del mundo a la cámara...



Dado un punto en el mundo P_w , queremos proyectarlo en la imagen.

De coordenadas del mundo a la cámara...



Dado un punto en el mundo P_w , queremos proyectarlo en la imagen.

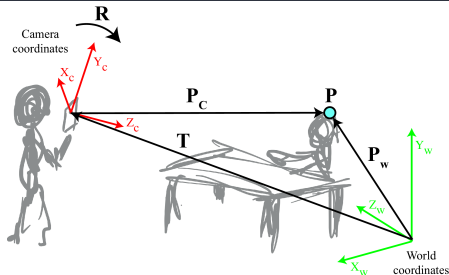
Expresar el punto en el sistema de la cámara:

- ▶ Primero, trasladar el sistema de coordenadas
- ▶ Luego, rotar para alinear con la cámara

$$P_c = R(P_w - T) = RP_w - RT$$

- ▶ R : matriz de rotación (3×3)
- ▶ T : posición de la cámara en el mundo, T mueve el origen al centro de la cámara
- ▶ R alinea los ejes

Transformación homogénea (extrínsecos)



Usamos coordenadas homogéneas:

Traslación:

$$M_1 = \begin{bmatrix} 1 & 0 & 0 & -T_x \\ 0 & 1 & 0 & -T_y \\ 0 & 0 & 1 & -T_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

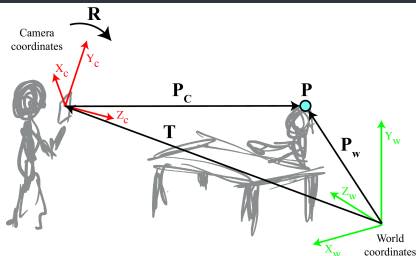
Rotación:

$$M_2 = \begin{bmatrix} R_1 & R_2 & R_3 & 0 \\ R_4 & R_5 & R_6 & 0 \\ R_7 & R_8 & R_9 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Transformación completa:

$$P_c = M_2 M_1 P_w$$

Parámetros extrínsecos de la cámara



$$P_c = M_2 M_1 P_w^4$$

Forma explícita:

$$M_2 M_1 = \begin{bmatrix} R_1 & R_2 & R_3 & -r_1 T \\ R_4 & R_5 & R_6 & -r_2 T \\ R_7 & R_8 & R_9 & -r_3 T \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} R & -RT \\ \mathbf{0}^T & 1 \end{bmatrix}$$

Transformación final:

$$P_c = \begin{bmatrix} R & -RT \\ \mathbf{0}^T & 1 \end{bmatrix} P_w$$

- **R**: orientación de la cámara
- **T**: posición de la cámara en el mundo
- **-RT**: traslación en el sistema de la cámara
- **r_i representa la fila i de **R****

⁴ $M_2 M_1$ tiene ultima fila con ceros para no afectar la parte espacial, y el 1 es la coord. homogenea

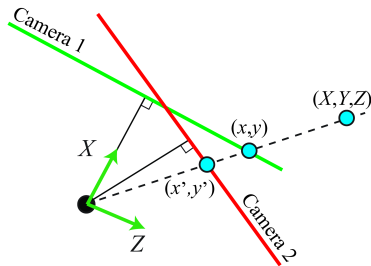
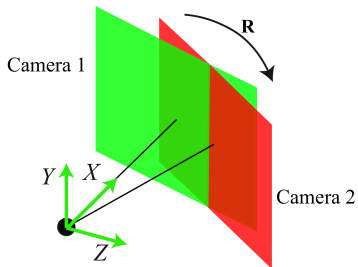
Camera Rotation



- ▶ Imágenes capturadas desde el mismo punto (sin traslación)
- ▶ Solo existe rotación de la cámara
- ▶ El centro de la cámara permanece fijo
- ▶ La relación entre imágenes es una homografía
- ▶ Válido para todos los puntos, independiente de su profundidad

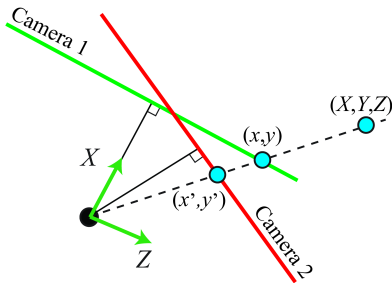
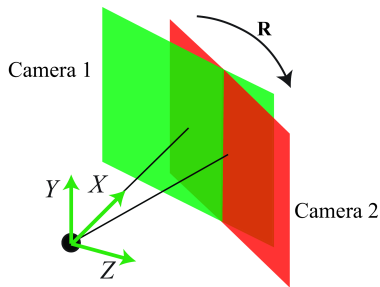
Camera Rotation

- Consideramos un punto 3D $\mathbf{P} = [X, Y, Z]^T$



- Dos cámaras observan el mismo punto:
No hay traslación, solo rotación entre cámaras.

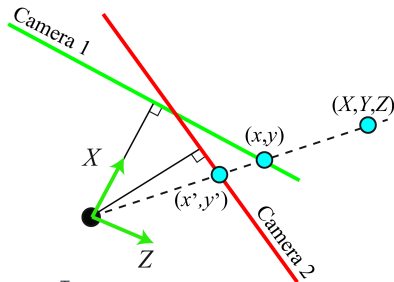
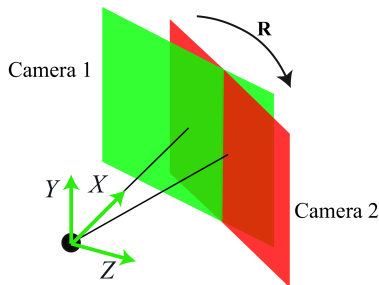
Proyección en cámara 1



- ▶ Alineamos el sistema mundo con la cámara 1
- ▶ Se define el sistema de coordenadas del mundo alineado con la cámara 1:
 - Rotación: $\mathbf{R} = \mathbf{I}$
 - Traslación: $\mathbf{t} = \mathbf{0}$
- ▶ La proyección de un punto 3D $\mathbf{P} = [X, Y, Z]^T$ es:

$$\lambda_1 \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

Proyección en cámara 1



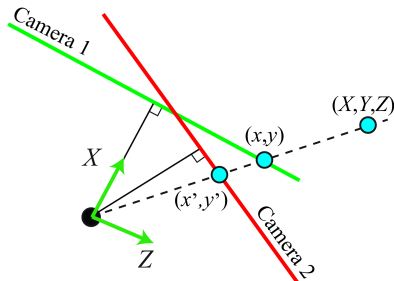
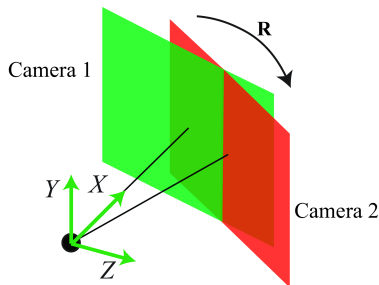
► La proyección de un punto 3D $\mathbf{P} = [X, Y, Z]^T$ es:

$$\lambda_1 \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

- $\mathbf{I}$: matriz identidad 3×3 (no hay rotación)
- $\mathbf{K}$: matriz intrínseca de la cámara

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Proyección en cámara 2

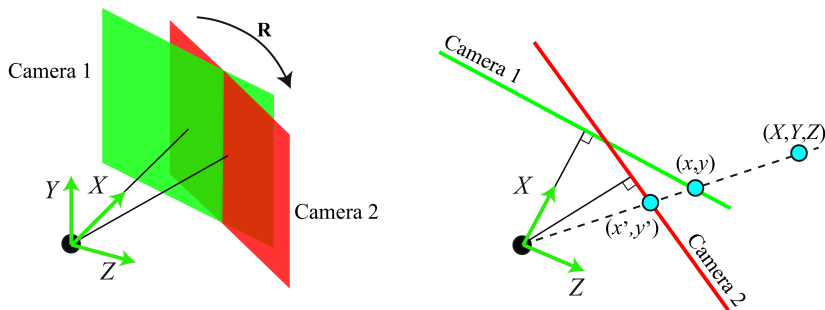


- ▶ Ahora el mismo punto 3D $\mathbf{P} = [X, Y, Z]^T$ es observado desde la **cámara 2**
- ▶ Se considera:
 - Rotación: $\mathbf{R}$
 - Traslación: $\mathbf{t} = \mathbf{0}$
- ▶ Proyección en la imagen:

$$\lambda_2 \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{0} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \mathbf{KR} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

- A diferencia de la cámara 1, ahora sí hay rotación

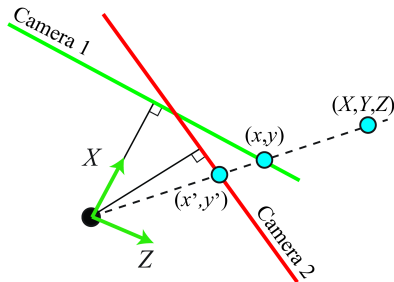
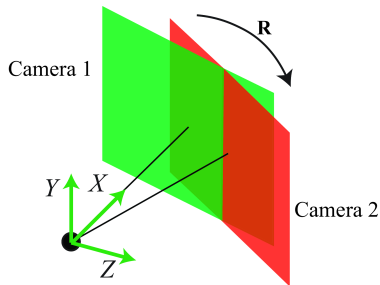
Relación entre ambas cámaras



- ▶ Ambas cámaras observan el mismo punto 3D: $\mathbf{P} = [X, Y, Z]^T$
- ▶ Invertiendo las proyecciones:

$$\mathbf{P} = \lambda_1 \mathbf{K}^{-1} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \lambda_2 \mathbf{R}^{-1} \mathbf{K}^{-1} \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$
$$\frac{\lambda_2}{\lambda_1} \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{K} \mathbf{R} \mathbf{K}^{-1} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Relación entre ambas camaras (2)



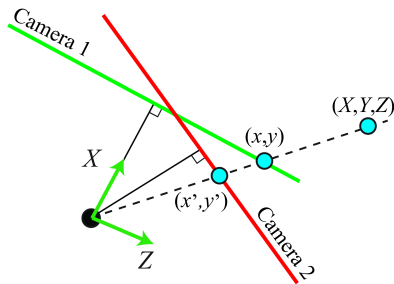
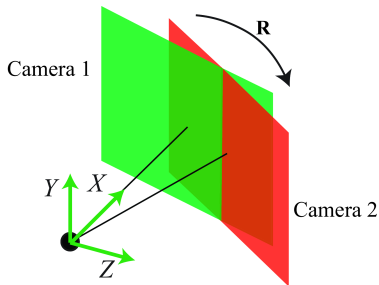
► Forma final (homografía):

$$\lambda \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad \text{con} \quad \mathbf{H} = \mathbf{K} \mathbf{R} \mathbf{K}^{-1}$$

- La rotación entre cámaras induce una homografía
- No hay traslación, mismo centro de proyección

Interpretación geométrica

- ▶ La cámara rota pero no se traslada
- ▶ El centro de proyección es el mismo
- ▶ Cada rayo proyectivo se mantiene
- ▶ Por lo tanto, no importa la profundidad Z , y la relación es puramente 2D



Direct Linear Transform (DLR) Algorithm

Direct Linear Transform (DLR) Algorithm

Si conocemos las coordenadas de varios puntos 3D de la escena en coordenadas del mundo y también conocemos las posiciones en las que esos puntos 3D se proyectan en la imagen.

¿Podemos utilizarlas para recuperar los parámetros intrínsecos y extrínsecos de la cámara?

Direct Linear Transform (DLR) Algorithm

Si conocemos las coordenadas de varios puntos 3D de la escena en coordenadas del mundo y también conocemos las posiciones en las que esos puntos 3D se proyectan en la imagen.

¿Podemos utilizarlas para recuperar los parámetros intrínsecos y extrínsecos de la cámara?

- ▶ Conocemos: los puntos 3D en el mundo: $\mathbf{P}_i$ y sus proyecciones en la imagen $\mathbf{p}_i$.
- ▶ Se espera estimar:
 - Matriz de proyección $\mathbf{M}$
 - $\mathbf{M}$ es equivalente a parámetros intrínsecos + extrínsecos

$$\mathbf{p}_i = \mathbf{M} \mathbf{P}_i$$

Coordenadas homogeneas de p .

$$\mathbf{p}_i = \mathbf{M} \mathbf{P}_i = \begin{bmatrix} \mathbf{m}_0^T \\ \mathbf{m}_1^T \\ \mathbf{m}_2^T \end{bmatrix} \mathbf{P}_i$$

$$x_i = \frac{\mathbf{m}_0^T \mathbf{P}_i}{\mathbf{m}_2^T \mathbf{P}_i}, \quad y_i = \frac{\mathbf{m}_1^T \mathbf{P}_i}{\mathbf{m}_2^T \mathbf{P}_i}$$

La proyección es una **transformación proyectiva** (no lineal en coordenadas euclidianas, pero lineal en homogéneas).

Direct Linear Transform (DLR) Algorithm (2)

Reordenando:

$$\mathbf{P}_i^T \mathbf{m}_0 - x_i \mathbf{P}_i^T \mathbf{m}_2 = 0$$

$$\mathbf{P}_i^T \mathbf{m}_1 - y_i \mathbf{P}_i^T \mathbf{m}_2 = 0$$

Escrito en forma matricial:

$$\begin{bmatrix} -\mathbf{P}_1^T & 0 & x_1 \mathbf{P}_1^T \\ 0 & -\mathbf{P}_1^T & y_1 \mathbf{P}_1^T \\ \vdots & \vdots & \vdots \\ -\mathbf{P}_N^T & 0 & x_N \mathbf{P}_N^T \\ 0 & -\mathbf{P}_N^T & y_N \mathbf{P}_N^T \end{bmatrix} \begin{bmatrix} \mathbf{m}_0 \\ \mathbf{m}_1 \\ \mathbf{m}_2 \end{bmatrix} = 0$$

$$\mathbf{A} \mathbf{m} = 0$$

Cada punto genera 2 ecuaciones lineales, por lo que, se necesitan al menos 6 correspondencias para resolver los 12 parámetros de M...

Direct Linear Transform (DLR) Algorithm (3)

Se puede resumir como:

$$\mathbf{A}\mathbf{m} = 0,$$

donde $\mathbf{A}$ es la matriz $2N \times 12$ y $\mathbf{m}$ es un vector con los elementos de $\mathbf{M}^5$

- $\mathbf{M}$ (3×4) tiene 11 grados de libertad, dado que la escala global no importa.

⁵ $2N$ porque hay 2 ecuaciones por correspondencia de puntos (una para x y otra para y), y todas las ecuaciones tienen 12 incógnitas

Direct Linear Transform (DLR) Algorithm (3)

Se puede resumir como:

$$\mathbf{A}\mathbf{m} = 0,$$

donde $\mathbf{A}$ es la matriz $2N \times 12$ y $\mathbf{m}$ es un vector con los elementos de $\mathbf{M}^5$

► $\mathbf{M}$ (3×4) tiene 11 grados de libertad, dado que la escala global no importa.

En la práctica NO existe solución exacta: $\mathbf{A}\mathbf{m} = 0$, en cambio, se busca minimizar $\|\mathbf{A}\mathbf{m}\|^2$, lo cual, se obtiene como el eigenvector asociado al eigenvalue mas pequeño de la matriz $\mathbf{A}^T \mathbf{A}$. Generalmente, se resuelve usando SVD (Singular Value Decomposition)

⁵_{2N} porque hay 2 ecuaciones por correspondencia de puntos (una para x y otra para y), y todas las ecuaciones tienen 12 incógnitas

Direct Linear Transform (DLR) Algorithm (3)

Se puede resumir como:

$$\mathbf{A}\mathbf{m} = 0,$$

donde $\mathbf{A}$ es la matriz $2N \times 12$ y $\mathbf{m}$ es un vector con los elementos de $\mathbf{M}^5$

► $\mathbf{M}$ (3×4) tiene 11 grados de libertad, dado que la escala global no importa.

En la práctica NO existe solución exacta: $\mathbf{A}\mathbf{m} = 0$, en cambio, se busca minimizar $\|\mathbf{A}\mathbf{m}\|^2$, lo cual, se obtiene como el eigenvector asociado al eigenvalue mas pequeño de la matriz $\mathbf{A}^T\mathbf{A}$. Generalmente, se resuelve usando SVD (Singular Value Decomposition)

Resumen de DLR

1. Recolectar correspondencias 3D-2D
2. Construir matriz $\mathbf{A}$
3. Resolver $\mathbf{A}\mathbf{m} = 0$
4. Reorganizar los terminos de $\mathbf{m}$ en $\mathbf{M}$

► Obtener $\mathbf{M}$ es el primer paso para calibrar una cámara. Luego se separa en:

$$\mathbf{M} = \mathbf{K}[\mathbf{R}|\mathbf{t}]$$

(ver Sección 39.7.2 del libro guía)

⁵2N porque hay 2 ecuaciones por correspondencia de puntos (una para x y otra para y), y todas las ecuaciones tienen 12 incógnitas

Random Sampling with Consensus (RANSAC)



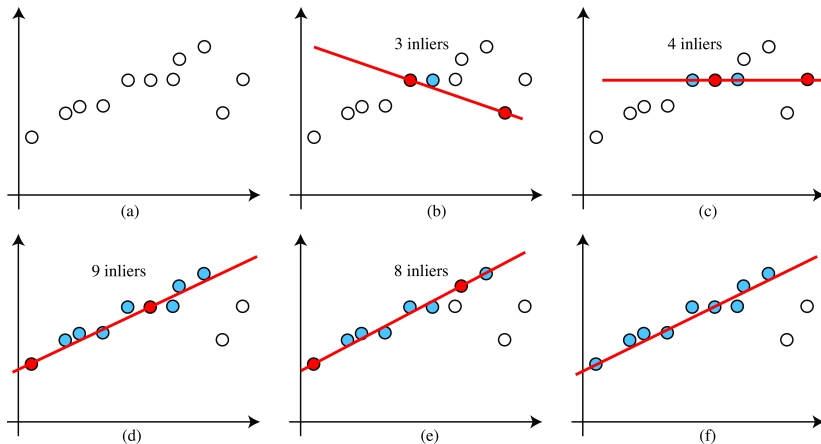
- Las correspondencias 2D-3D pueden contener errores (outliers)
- Por ejemplo, matches incorrectos en imágenes

Random Sampling with Consensus (RANSAC)

- ▶ Selecciona subconjuntos aleatorios de datos
- ▶ Ajusta un modelo (ej: homografía o matriz $\mathbf{M}$)
- ▶ Evalúa cuántos puntos lo apoyan (**inliers**)
- ▶ Repite múltiples veces

El mejor modelo es el que maximiza el número de inliers

Robust Model Fitting: Random Sampling with Consensus (RANSAC)



(a) Puntos de datos para los que se busca una estimación robusta de la línea. (b) Se seleccionan al azar dos puntos (en azul), suficientes para estimar los parámetros del modelo de la línea. Se calcula el número de valores válidos para este modelo. (c - e) Se repite el proceso varias veces. (f) Por último, se selecciona el modelo con el mayor número de valores válidos.

Estimación de homografía H :

- ▶ Se usa DLT (Direct Linear Transform), y se requieren 8 ecuaciones.
- ▶ Dado que cada correspondencia aporta 2 ecuaciones, se necesitan al menos 4 puntos

RANSAC:

- ▶ Seleccionar aleatoriamente 4 correspondencias
- ▶ Estimar H con DLT
- ▶ Evaluar inliers
- ▶ Repetir múltiples veces

RANSAC permite estimar H de forma robusta frente a outliers.

Image Stitching

Using DLT and RANSAC with can compute the homography between each pair of images in figure and align them with respect to a reference image (in this example we pick the middle image as a reference).



ORB-SLAM3

ORB-SLAM3 is the first real-time SLAM library able to perform Visual, Visual-Inertial and Multi-Map SLAM with monocular, stereo and RGB-D cameras, using pin-hole and fisheye lens models. In all sensor configurations, ORB-SLAM3 is as robust as the best systems available in the literature, and significantly more accurate.

DOG-SLAM: Enhancing Dynamic Visual SLAM Precision Through GMM-Based Dynamic Object Removal and ORB-Boost

Jiming Long[✉], Fei Wang[✉], *Member, IEEE*, Mingyang Liu[✉], Yu Wang[✉], and Qiang Zou[✉]

Abstract—Accurate mapping and navigation in dynamic environments pose substantial challenges for autonomous robotics. Traditional visual simultaneous localization and mapping (Visual SLAM) methods frequently encounter difficulties with dynamic elements, leading to data association errors and reduced localization accuracy. This study introduces DOG-SLAM, a dynamic Visual SLAM system designed to enhance performance in dynamic environments by addressing three critical aspects: more precise detection of dynamic regions, extraction of an adequate number of static feature points, and improvement of static feature point quality. DOG-SLAM incorporates a dynamic object removal module and a descriptor enhancement module. The dynamic object removal module uses a pseudosemantic segmentation strategy based on GMM, combined with RGB and depth data, for precise segmentation of dynamic regions, thus preserving a greater number of static regions. In addition, a dynamic preremoval strategy is implemented to extract a sufficient number of static feature points, maintaining feature point stability and improving localization accuracy. The descriptor enhancement module uses FeatureBooster to generate ORBboost descriptors, thereby improving the robustness and match rate of static feature points. Experimental validation on standard datasets, including TUM and BONN, demonstrated that DOG-SLAM reduced absolute trajectory error (ATE) by up to 98.38% and 99.26%, respectively, compared with its base model ORB-SLAM3. Moreover, our approach led the field in positioning accuracy compared with current state-of-the-art dynamic SLAM technologies. These results highlight the improved accuracy of

themselves while generating detailed, real-time maps of their surroundings. Although traditional Visual SLAM methods, such as ORB-SLAM2 [1], ORB-SLAM3 [2], and VINS-Mono [3], perform well in static environments, the presence of dynamic elements in real-world scenarios can lead to data association errors, affecting localization and mapping accuracy.

To address these challenges in dynamic environments, dynamic Visual SLAM technologies have been developed. For dynamic Visual SLAM systems, the quantity and quality of static feature points directly influence localization and mapping accuracy. In dynamic environments, obtaining sufficient high-quality static feature points is paramount, which involves three key aspects: 1) finer detection of dynamic regions; 2) extraction of sufficient static feature points in static regions; and 3) enhancement of the quality of the obtained static feature points.

- 1) Currently, many dynamic Visual SLAM systems use deep-learning-based object detection techniques to identify dynamic objects, such as Zhang et al. [4], YOLO-SLAM [5], and SG-SLAM [6]. These methods significantly improve the stability and accuracy of Visual SLAM systems in dynamic environments



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Projective Geometry

Doctorado en Inteligencia Artificial

Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile